

מדריך מנגנוני הליבה המרוכז והמעודכן

UI_alfa - Core Executive Guide

ניתוח מעמיק וממוקד של מנגנוני הליבה: מסחר, אשראי, ניהול סיכונים, ניתוח
AI, גשר MT5 ומסדי נתונים

מטרת המסמך: תמצית מקיפה וממוקדת של 7 מנגנוני הליבה החשובים ביותר במערכת

טכנולוגיות: Java 17, Spring Boot 3, React (Vite), MT5 Bridge, Binance API, AI Predictor

תאריך עדכון: יולי 2026

1. פרק 1: סקירה ארכיטקטונית ומבנה השרת
2. פרק 2: מנגנון הזדהות, אבטחה ו-4 תרחישי הדמו
3. פרק 3: מנגנון קנייה ומכירה של נכסים וביצוע הוראות
4. פרק 4: מנגנון הקרדיט ליין, ניהול סיכונים ומנוע הנזלה כפויה
5. פרק 5: מנגנון המאצ'ינג, ניתוב AI וגשר MetaTrader 5
6. פרק 6: מבנה מסד הנתונים ונעילת טרנזקציות למניעת תחרות
7. פרק 7: ממשק אדמין והזרמת מחירי שוק בלייב

פרק 1: סקירה ארכיטקטונית ומבנה השרת

מערכת **UI_alfa** הינה פלטפורמת מסחר וברוקראז' אלגוריתמית מקצה לקצה. המערכת מאפשרת מסחר בזמן אמת במט"ח, קריפטו ומניות, תוך מתן מסגרות אשראי, מינוף (Margin Trading), ניתוב מונחה בינה מלאכותית, וסנכרון מול שרתי MetaTrader 5.

ארכיטקטורה מודולרית: ה-Backend תוכנן ב-Java 17 וב-Spring Boot 3. ה-Frontend בנוי ב-React ב-Vite עם Tailwind CSS וגרפים אינטראקטיביים של TradingView.

1.1 אתחול השרת (BackendApplication.java)

נקודת הכניסה לשרת ה-Backend מפעילה את Spring Boot ומגדירה את מנגנוני התזמון המחזוריים (:@EnableScheduling)

```
package com.brokerui;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.scheduling.annotation.EnableScheduling;

@SpringBootApplication
@EnableScheduling
public class BackendApplication {
    public static void main(String[] args) {
        SpringApplication.run(BackendApplication.class, args);
    }
}
```

1.2 אבטחה, CSRF ואישור בקשות (SecurityConfig.java)

מנגנון האבטחה מגדיר את הגנת ה-CSRF מבוססת העוגיות, הגדרות CORS, והרשאות נתיבים לפי תפקידי משתמש (ROLE_ADMIN / ROLE_USER):

```

@Configuration
@EnableWebSecurity
public class SecurityConfig {
    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .cors(c -> c.configurationSource(corsConfigurationSource()))
            .csrf(csrf -> csrf
                .csrfTokenRepository(CookieCsrfTokenRepository.withHttpOnlyFalse())
                .ignoringRequestMatchers("/api/auth/register", "/api/broker/**")
            )
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/auth/register", "/api/auth/csrf", "/api/health").permitAll()
                .requestMatchers("/api/admin/**").hasRole("ADMIN")
                .anyRequest().authenticated()
            );

        return http.build();
    }
}

```

פרק 2: מנגנון הזדהות, אבטחה ו-4 תרחישי הדמו

המערכת מפרידה בצורה מלאה בין **משתמשים רגילים (Real Accounts)** לבין **משתמשי דמו (Demo Simulators)**:

- **משתמש רגיל:** נרשם ב-`/api/auth/register` (`AuthApiController.java`), הסיסמה מוצפנת ב-`BCrypt`, ונדרש אימות זהות KYC (`KycCase.java`).
- **משתמש דמו:** מועלה אוטומטית בהפעלת השרת ב-`AuthBootstrap.java` עם 4 תרחישי מסחר מובנים.

2.1 תהליך רישום משתמשים (`AuthApiController.java`)

```
@RestController
public class AuthApiController {
    @PostMapping("/api/auth/register")
    @Transactional
    public ResponseEntity<> register(@Valid @RequestBody RegisterRequest req) {
        String email = req.email().trim().toLowerCase(Locale.ROOT);
        if (repo.existsByEmailIgnoreCase(email)) {
            return ResponseEntity.status(HttpStatus.CONFLICT)
                .body(Map.of("ok", false, "error", "email_taken"));
        }
        AppUser u = new AppUser();
        u.setEmail(email);
        u.setDisplayName(req.displayName().trim());
        u.setPasswordHash(encoder.encode(req.password()));
        u.setRole(UserRole.USER);
        repo.save(u);
        return ResponseEntity.status(HttpStatus.CREATED).body(UserDto.from(u));
    }
}
```

2.2 4 תרחישי הדמו המובנים ב-`AuthBootstrap.java`

שם המשתמש	דוא"ל	מאפיינים ותרחיש נבדק
Default Demo Trader	<code>demo@broker.local</code>	חשבון דמו סטנדרטי עם יתרה של \$10,000 ומסגרת אשראי של \$10,000.
Hedging Trader	<code>hedge@broker.local</code>	מסחר מגודר: מחזיק פוזיציות מקבילות (LONG ו-SHORT) על זהב וביטקוין.
VIP Trader	<code>vip@broker.local</code>	חשבון VIP בעל מסגרת אשראי של \$25,000 וריבית יומית מופחתת (0.3% ביום).
High Risk Trader	<code>highrisk@broker.local</code>	חשבון בסיכון גבוה עם חוב אשראי פעיל, לבדיקת מנגנון ה-Margin Call וההנזלה.

```

@Component
public class AuthBootstrap implements ApplicationRunner {
    @Override
    public void run(ApplicationArguments args) {
        AppUser demoUser = repo.findByEmailIgnoreCase("demo@broker.local")
            .orElseGet(() -> {
                AppUser demo = new AppUser();
                demo.setEmail("demo@broker.local");
                demo.setDisplayName("Demo Trader");
                demo.setPasswordHash(encoder.encode("demo123"));
                demo.setRole(UserRole.USER);
                return repo.save(demo);
            });

        if (accountRepo.findFirstByUserIdOrderByIdAsc(demoUser.getId()).isEmpty()) {
            TradingAccount acc = new TradingAccount();
            acc.setUser(demoUser);
            acc.setAccountType("DEMO");
            acc.setBalance(new BigDecimal("10000.00"));
            acc.setCreditLimit(new BigDecimal("10000.00"));
            acc.setLeverage(100);
            accountRepo.save(acc);
        }
    }
}

```

פרק 3: מנגנון קנייה ומכירה של נכסים וביצוע הוראות

מנגנון הליבה של המסחר מנוהל ב-OrderExecutionService.java ו ב-BrokerApiController.java. המערכת תומכת בהוראות MARKET, LIMIT ו-STOP.

Margin Required = (Execution Price × Quantity × Contract Size) / Leverage

Commission Fee = CalculateCommission(Symbol, Quantity, Execution Price)

3.1 קוד מפתח: מנוע ביצוע הוראות מסחר (OrderExecutionService.java)

```
@Scheduled(fixedDelay = 2000)
public void tick() {
    List<BrokerOrder> open = orderRepo.findTop50ByStatusOrderByCreatedAtAsc("NEW");
    for (BrokerOrder o : open) {
        try {
            self.tryExecute(o.getId());
        } catch (Exception ignored) {}
    }
}

@Transactional
public void tryExecute(Long orderId) {
    BrokerOrder order = orderRepo.findById(orderId).orElse(null);
    if (order == null || !"NEW".equalsIgnoreCase(order.getStatus())) return;

    double price = priceService.getLivePrice(order.getSymbolCode());
    if (price <= 0) return;
    BigDecimal last = BigDecimal.valueOf(price);

    String side = order.getSide().toUpperCase();
    String type = order.getOrderType().toUpperCase();

    boolean shouldFill = (type.equals("MARKET")) ||
        switch (type) {
            case "LIMIT" -> shouldFillLimit(order, side, last);
            case "STOP" -> shouldFillStop(order, side, last);
            default -> false;
        };

    if (shouldFill) {
        executeFilled(order, last);
    }
}

private void executeFilled(BrokerOrder order, BigDecimal rawPrice) {
    TradingAccount ta = accountRepo.findByIdForUpdate(order.getTradingAccount().getId()).orElseThrow();
    boolean isBuy = "BUY".equalsIgnoreCase(order.getSide());
    BigDecimal price = TradingFees.applySpread(rawPrice, isBuy);
    BigDecimal qty = order.getQuantity();

    // חישוב עמלת מסחר וחיוב
    BigDecimal commission = TradingFees.calculateCommission(order.getSymbolCode(), qty, price);
    TradingFees.charge(ta, order, commission);

    // פתיחת פוזיציה בביטחונות
    Position pos = openNewPosition(ta, order.getSymbolCode(), isBuy ? "LONG" : "SHORT", qty, price, order);
    positionRepo.save(pos);

    order.setStatus("FILLED");
    order.setFilledAt(Instant.now());
    order.setEntryPrice(price);
}
```

```
orderRepo.save(order);  
}
```

פרק 4: מנגנון הקרדיט ליין, ניהול סיכונים ומנוע הנזלה

כפוייה

מנגנון האשראי ב-MarginLoanService.java מעניק אשראי בטחונות אוטומטי בסגנון Bybit.

$$\text{Margin Level} = \text{Live Equity} / \text{Borrowed Balance}$$

4.1 קוד מפתח: לקיחת אשראי והחזר חוב אוטומטי (MarginLoanService.java)

```
public boolean tryCoverShortfall(TradingAccount ta, BigDecimal required, AppUser user, HttpServletRequest req) {
    if (required.compareTo(BigDecimal.ZERO) <= 0) return true;

    if (ta.getBalance().compareTo(required) >= 0) {
        ta.setBalance(ta.getBalance().subtract(required));
        return true;
    }

    BigDecimal shortfall = required.subtract(ta.getBalance());
    BigDecimal currentDebt = ta.getBorrowedBalance() == null ? BigDecimal.ZERO : ta.getBorrowedBalance();

    if (currentDebt.add(shortfall).compareTo(ta.getCreditLimit()) > 0) {
        return false; // חריגה ממסגרת הקרדיט - ההוראה נדחתה
    }

    ta.setBalance(BigDecimal.ZERO);
    ta.setBorrowedBalance(currentDebt.add(shortfall));
    writeLedger(ta, "BORROW", shortfall, "Auto-borrow to cover trade shortfall");
    return true;
}

public void repaySettlementOrBorrow(TradingAccount ta, BigDecimal amount) {
    if (amount == null || amount.compareTo(BigDecimal.ZERO) == 0) return;

    if (amount.compareTo(BigDecimal.ZERO) > 0) {
        BigDecimal debt = ta.getBorrowedBalance() == null ? BigDecimal.ZERO : ta.getBorrowedBalance();
        if (debt.compareTo(BigDecimal.ZERO) > 0) {
            BigDecimal repay = amount.min(debt);
            ta.setBorrowedBalance(debt.subtract(repay));
            BigDecimal remainder = amount.subtract(repay);
            ta.setBalance(ta.getBalance().add(remainder));
            writeLedger(ta, "REPAY", repay, "Auto-repay from trade settlement");
        } else {
            ta.setBalance(ta.getBalance().add(amount));
        }
    }
}
```

4.2 מנוע הנזלה כפויה בלופ מתזמן (Liquidation Engine)

```
@Scheduled(fixedDelay = 10000)
@Transactional
public void checkLiquidations() {
    List<TradingAccount> debtors = accountRepo.findByBorrowedBalanceGreaterThan(BigDecimal.ZERO);
    for (TradingAccount ta : debtors) {
        BigDecimal level = computeMarginLevel(ta);
        if (level == null) continue;

        if (level.compareTo(LIQUIDATION_LEVEL) < 0) {
            liquidateAccount(ta, "Margin level fell below 100% liquidation threshold");
        }
    }
}
```

פרק 5: מנגנון המאצ'ינג, ניתוב AI וגשר MetaTrader 5

מנוע הניתוב משלב מודל בינה מלאכותית (NNPredictorClient.java) המחליץ על ניתוב B-Book (פנימי) מול A-Book (חיצוני), וגשר תקשורת מול שרת MetaTrader 5 ב-MT5IntegrationService.java.

5.1 קוד מפתח: ניתוב AI ושידור ל-MT5 Bridge

```
// AI-מודל ה (Features Vector) הכנת וקטור המאפיינים
double[] features = {
    buyQtyNorm, sellQtyNorm, spreadNorm, imbalance,
    midPriceNorm, bookDepthBuy, bookDepthSell, 0.72
};

Map<String, Object> pred = nnPredictorClient.getPrediction(features);
if (pred != null) {
    double routeRecommendation = ((Number) pred.get("routeRecommendation")).doubleValue();
    order.setNnRouteRecommendation(routeRecommendation > 0.5 ? "INTERNAL" : "EXTERNAL");

    if (routeRecommendation <= 0.5) {
        // MetaTrader 5 ניתוב חיצוני לגשר
        mt5Service.sendOrderToMT5(order);
    }
}
```

5.2 גשר & MT5IntegrationService.java & MT5ConnectionManager.java

```
@Service
public class MT5IntegrationService {
    private final MT5ConnectionManager connectionManager;

    public MT5IntegrationService(MT5ConnectionManager connectionManager) {
        this.connectionManager = connectionManager;
    }

    public void sendOrderToMT5(BrokerOrder order) {
        String payload = String.format("ORDER|%s|%s|%.4f|%.4f",
            order.getSymbolCode(), order.getSide(), order.getQuantity().doubleValue(), order.getEntryPrice());
        connectionManager.send(payload);
    }
}
```

פרק 6: מבנה מסד הנתונים ונעילת טרנזקציות למניעת תחרות

שכבת הנתונים משתמשת ב-JPA, Spring Data ובעילה קפדנית (Pessimistic Write Locking) ב-2. TradingAccountRepository.java למניעת Race Conditions במסחר מקבילי.

6.1 נעילת טרנזקציות ב-TradingAccountRepository.java

```
public interface TradingAccountRepository extends JpaRepository<TradingAccount, Long> {  
    @Lock(LockModeType.PESSIMISTIC_WRITE)  
    @Query("SELECT a FROM TradingAccount a WHERE a.id = :id")  
    Optional<TradingAccount> findByIdForUpdate(@Param("id") Long id);  
  
    List<TradingAccount> findByBorrowedBalanceGreaterThan(BigDecimal amount);  
}
```

6.2 ישות חשבון המסחר (TradingAccount.java)

```
@Entity  
@Table(name = "trading_accounts")  
public class TradingAccount {  
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)  
    private Long id;  
  
    @ManyToOne(fetch = FetchType.LAZY)  
    @JoinColumn(name = "user_id")  
    private AppUser user;  
  
    private String accountType; // DEMO / REAL  
    private BigDecimal balance;  
    private BigDecimal equity;  
    private BigDecimal borrowedBalance; // חוב אשראי אקטיבי  
    private BigDecimal creditLimit; // תקרת אשראי  
    private BigDecimal dailyInterestRate;  
    private Integer leverage;  
}
```

פרק 7: ממשק אדמין והזרמת מחירי שוק בלייב

ה-Controller ב-AdminTradeController.java מספק מגוון יכולות שליטה לפאנל האדמין)
(AdminDashboard_Local.html, כולל הרצת ריביות והנזלות ידניות.

7.1 קוד מפתח: פאנל אדמין (AdminTradeController.java)

```
@RestController
@RequestMapping("/api/admin")
public class AdminTradeController {
    private final MarginLoanService marginLoanService;

    @PostMapping("/run-interest")
    public ResponseEntity<?> runInterestNow() {
        marginLoanService accrueDailyInterest();
        return ResponseEntity.ok(Map.of("ok", true));
    }

    @PostMapping("/run-liquidation")
    public ResponseEntity<?> runLiquidationCheckNow() {
        marginLoanService.checkLiquidations();
        return ResponseEntity.ok(Map.of("ok", true));
    }
}
```

7.2 מנגנון הזרמת מחירי שוק בלייב (MarketPriceService.java)

```
@Service
public class MarketPriceService {
    private final BinancePriceService binanceService;
    private final YahooFinanceService yahooService;

    public MarketPriceService(BinancePriceService binanceService, YahooFinanceService yahooService) {
        this.binanceService = binanceService;
        this.yahooService = yahooService;
    }

    public double getLivePrice(String symbol) {
        if (symbol.endsWith("USDT") || symbol.startsWith("BTC")) {
            return binanceService.getPrice(symbol);
        }
        return yahooService.getPrice(symbol);
    }
}
```

